Experiment 1: Explore Python Libraries for Deep Learning (TensorFlow & Keras)

Aim: *To explore Python libraries for deep learning (e.g., TensorFlow, Keras).*

Q1. What are the key features of the TensorFlow library? (Pre-Lab)

Ans: TensorFlow is an open-source deep learning library developed by Google for scalable numerical computation and model training with automatic differentiation, multi-device (CPU/GPU/TPU) support, and high-level APIs.

Q2. What applications can be implemented using TensorFlow and Keras? (Post-Lab)

Ans: Image classification, object detection, natural language processing (NLP), sentiment analysis, speech recognition, image compression (autoencoders), time-series forecasting, and generative AI.

Q3. What is the difference between TensorFlow and Keras?

Ans: TensorFlow is a complete end-to-end framework providing both low-level tensor operations and production deployment tools, whereas Keras (tf.keras) is a high-level neural networks API running on top of TensorFlow for rapid model prototyping.

Q4. What is a Tensor in Deep Learning?

Ans: A tensor is a multi-dimensional array generalized to any number of dimensions: a 0D tensor is a scalar, 1D is a vector, 2D is a matrix, and 3D/4D tensors represent sequences or batches of images.

Q5. What is the difference between Sequential API and Functional API in Keras?

Ans: Sequential API builds models layer-by-layer in a single linear stack, while the Functional API allows building complex architectures with multiple inputs/outputs, shared layers, and residual connections.

Q6. What does model.compile() do in Keras?

Ans: model.compile() configures the model for training by specifying the optimizer (e.g., Adam, SGD), the loss function (e.g., binary_crossentropy, mse), and evaluation metrics (e.g., accuracy).

Q7. What is the role of model.fit() and model.evaluate()?

Ans: model.fit() trains the model on training data for a fixed number of epochs, whereas model.evaluate() computes the loss and metrics on unseen test data without updating weights.

Q8. What is Eager Execution in TensorFlow 2.x?

Ans: Eager execution evaluates operations immediately as they are called from Python without building a static computational graph first, making debugging and tensor inspection intuitive.

Experiment 2: Implement McCulloch-Pitts Model for Binary Logic Functions

Aim: *Implement McCulloch-Pitts (MP) neuron model for binary logic functions (AND, OR, NOT, NOR, NAND).*

Q1. Explain the McCulloch-Pitts (MP) neuron model. (Pre-Lab)

Ans: Proposed in 1943, the McCulloch-Pitts neuron is the earliest mathematical model of a biological neuron. It takes binary inputs (0 or 1), computes an unweighted/fixed-weight sum, and outputs 1 if the sum meets or exceeds a fixed threshold (theta), else 0.

Q2. Explain the simulation of an AND gate using the McCulloch-Pitts model. (Post-Lab)

Ans: For a 2-input AND gate, set weights w1 = 1, w2 = 1, and threshold theta = 2. Only when both inputs x1 = 1 and x2 = 1 does the net input y_in = x1*w1 + x2*w2 = 2 >= theta, producing output Y = 1.

Q3. How do excitatory and inhibitory inputs differ in an MP neuron?

Ans: Excitatory inputs have positive weights (w > 0) and increase the net sum toward the threshold. Inhibitory inputs have negative weights (p < 0) and prevent the neuron from firing if active.

Q4. What are the weights and threshold for an OR gate and a NOT gate in the MP model?

Ans: For a 2-input OR gate: w1 = 1, w2 = 1, threshold theta = 1. For a NOT gate: single inhibitory weight w = -1 and threshold theta = 0 (fires 1 when input x = 0, and 0 when x = 1).

Q5. Why can't a single McCulloch-Pitts neuron implement an XOR gate?

Ans: An XOR gate is not linearly separable—no single straight line (hyperplane) can separate the class-1 inputs (0,1) and (1,0) from the class-0 inputs (0,0) and (1,1).

Q6. What are the main limitations of the McCulloch-Pitts neuron?

Ans: 1) It only accepts binary inputs and produces binary outputs. 2) Weights and threshold must be set manually (no learning algorithm). 3) It can only solve linearly separable functions.

Q7. Write the mathematical activation rule of the McCulloch-Pitts neuron.

Ans: f(y_in) = 1 if y_in >= theta, else 0, where y_in = sum(w_i * x_i). This is a hard-limiting step (Heaviside) activation function.

Q8. How is an ANDNOT (X1 AND NOT X2) gate configured in an MP model?

Ans: Set w1 = 1 (excitatory), w2 = -1 (inhibitory), and threshold theta = 1. Output is 1 only when X1 = 1 and X2 = 0.

Experiment 3: Implement Perceptron Algorithm to Simulate Any Logic Gate

Aim: *Implement the Single-Layer Perceptron learning algorithm to simulate binary logic gates.*

Q1. What do you understand by a Perceptron Neuron? (Pre-Lab)

Ans: A Perceptron (introduced by Frank Rosenblatt in 1958) is a supervised binary classifier that computes a weighted sum of real-valued inputs, adds a learnable bias, and applies a step activation function.

Q2. What are the components of a Single-Layer Perceptron? (Post-Lab)

Ans: 1) Input layer (x1, x2, ..., xn), 2) Learnable synaptic weights (w1, w2, ..., wn), 3) Bias (b), 4) Net summation junction (z = w.x + b), and 5) Step/Sign activation function.

Q3. State the Perceptron Weight Update Rule.

Ans: When predicted output y_pred != target t: w_new = w_old + eta * (t - y_pred) * x, and b_new = b_old + eta * (t - y_pred), where eta is the learning rate.

Q4. How does the Rosenblatt Perceptron improve upon the McCulloch-Pitts neuron?

Ans: Unlike the MP neuron (which has manually fixed weights and binary-only inputs), the Perceptron accepts real-valued inputs and automatically learns weights and bias from labeled training data.

Q5. What is the Perceptron Convergence Theorem?

Ans: It states that if a training dataset is linearly separable, the Perceptron learning algorithm is guaranteed to converge to a separating hyperplane in a finite number of weight updates.

Q6. What is the role of the bias term (b) in a Perceptron?

Ans: The bias shifts the decision boundary (hyperplane) away from the origin, allowing the classifier to separate data even when all input features are zero.

Q7. What happens if the Perceptron algorithm is run on non-linearly separable data (like XOR)?

Ans: It will never converge; the weights will oscillate indefinitely unless a maximum epoch limit is enforced.

Q8. What is the role of the learning rate (eta) in Perceptron training?

Ans: Learning rate (0 < eta <= 1) controls the step size of weight adjustments during each misclassified sample update.

Experiment 4: Implement Multilayer Perceptron (MLP) Algorithm to Simulate XOR Gate

Aim: *Implement a Multilayer Perceptron (MLP) algorithm to simulate the non-linearly separable XOR gate.*

Q1. What is a Multilayer Perceptron (MLP)? (Pre-Lab)

Ans: A Multilayer Perceptron is a feedforward artificial neural network consisting of an input layer, one or more hidden layers, and an output layer, using non-linear activation functions and trained via backpropagation.

Q2. How does a Multilayer Perceptron simulate an XOR gate? (Post-Lab)

Ans: An MLP uses a hidden layer (with at least 2 hidden neurons) to transform the 2D input space into a new feature space (combining NAND and OR boundaries) where the XOR classes become linearly separable.

Q3. Why are non-linear activation functions necessary in hidden layers of an MLP?

Ans: Without non-linear activation functions (like Sigmoid, Tanh, or ReLU), a composition of multiple linear layers collapses into a single linear transformation, making the network no more powerful than a single-layer perceptron.

Q4. What is the truth table of the XOR gate and why is it non-linearly separable?

Ans: Inputs (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0. Plotted on a 2D plane, positive and negative points lie on opposite diagonals, requiring at least two decision boundaries.

Q5. Which activation function is used in the output layer for binary XOR classification?

Ans: The Sigmoid activation function sigma(z) = 1 / (1 + e^(-z)), which maps real numbers into the probability range (0, 1).

Q6. What is the Universal Approximation Theorem?

Ans: It states that a feedforward network with a single hidden layer containing a finite number of neurons and non-linear activations can approximate any continuous function on compact subsets of R^n.

Q7. What loss function is typically minimized when training an MLP on XOR?

Ans: Binary Cross-Entropy loss: L = -[y*log(y_hat) + (1-y)*log(1-y_hat)] or Mean Squared Error (MSE).

Q8. How many trainable parameters are in a 2-2-1 XOR MLP (including biases)?

Ans: Hidden layer: (2 inputs * 2 neurons) + 2 biases = 6. Output layer: (2 hidden * 1 output) + 1 bias = 3. Total = 9 trainable parameters.

Experiment 5: Apply Stochastic Gradient Descent (SGD) on a Single-Layer Feedforward Network

Aim: *Apply Stochastic Gradient Descent (SGD) to learn the parameters of a supervised single-layer feedforward neural network.*

Q1. Explain the use of the Gradient Descent algorithm in learning parameters. (Pre-Lab)

Ans: Gradient Descent iteratively updates network weights and biases in the opposite direction of the gradient of the loss function (w = w - eta * dL/dw) to find the parameter values that minimize error.

Q2. Explain the various types of Gradient Descent algorithms. (Post-Lab)

Ans: 1) Batch Gradient Descent (uses entire dataset per update), 2) Stochastic Gradient Descent / SGD (uses 1 sample per update), and 3) Mini-Batch Gradient Descent (uses a small batch k of samples per update).

Q3. How does Stochastic Gradient Descent (SGD) differ from Batch Gradient Descent?

Ans: Batch GD computes gradients over the entire training set before one weight update (slow for large data), whereas SGD updates weights after every single training example (fast, online updates).

Q4. Why does the training loss curve in SGD fluctuate (appear noisy)?

Ans: Because each weight update is computed from a single training sample rather than the true average gradient across all samples, causing high variance in step updates.

Q5. What is an advantage of the noise/fluctuation in SGD?

Ans: The stochastic noise helps the optimizer escape shallow local minima and saddle points in non-convex loss landscapes.

Q6. What happens if the learning rate in SGD is too high or too low?

Ans: If too high, the loss diverges or overshoots the minimum; if too low, training converges extremely slowly and may get stuck in suboptimal local minima.

Q7. What is Momentum in SGD?

Ans: Momentum accelerates SGD by accumulating an exponentially decaying moving average of past gradients, dampening oscillations and speeding up convergence along flat directions.

Q8. Write the mathematical weight update equation for SGD on sample i.

Ans: w := w - eta * grad_w L(f(x^(i); w), y^(i)), where (x^(i), y^(i)) is a single randomly shuffled training sample.

Experiment 6: Apply Mini-Batch Gradient Descent on a Single-Layer Feedforward Network

Aim: *Apply Mini-Batch Gradient Descent to learn the parameters of a supervised single-layer feedforward neural network.*

Q1. Explain the use of Gradient Descent in parameter optimization. (Pre-Lab)

Ans: Gradient descent optimizes neural network weights by computing partial derivatives of the cost function with respect to each weight and stepping downhill toward minimum loss.

Q2. Compare Batch, Stochastic, and Mini-Batch Gradient Descent. (Post-Lab)

Ans: Batch GD (batch = N): stable gradient, high memory/slow. SGD (batch = 1): fast updates, noisy trajectory. Mini-Batch GD (1 < batch < N, e.g., 32/64): balances fast vectorized GPU computation with stable convergence.

Q3. What is the difference between an Epoch and an Iteration (Step) in Mini-Batch GD?

Ans: An Epoch is one complete pass through the entire training dataset (N samples). An Iteration is a single weight update on one mini-batch (Iterations per epoch = N / batch_size).

Q4. Why are mini-batch sizes typically chosen as powers of 2 (e.g., 16, 32, 64, 128)?

Ans: Powers of 2 align efficiently with CPU/GPU memory word boundaries and SIMD/CUDA warp sizes, maximizing parallel matrix multiplication throughput.

Q5. How does Mini-Batch GD reduce variance compared to pure SGD?

Ans: Averaging gradients over B samples reduces the variance of the gradient estimate by a factor of 1/B, resulting in smoother and more reliable convergence toward the minimum.

Q6. Why should training data be shuffled before forming mini-batches each epoch?

Ans: Shuffling prevents the model from learning the order of samples and avoids biased gradients when data is grouped by class.

Q7. What adaptive optimizers build upon Mini-Batch Gradient Descent?

Ans: AdaGrad, RMSprop, and Adam (Adaptive Moment Estimation, which combines Momentum / 1st moment and RMSprop / 2nd moment).

Q8. If a dataset has 1,000 samples and batch_size = 50, how many weight updates occur in 10 epochs?

Ans: Updates per epoch = 1000 / 50 = 20 updates. Across 10 epochs, total weight updates = 20 * 10 = 200 updates.

Experiment 7: Implement Backpropagation Algorithm to Train a Deep Neural Network (DNN)

Aim: *Implement the Backpropagation algorithm to train a Deep Neural Network with at least two hidden layers.*

Q1. Explain the Backpropagation algorithm for training a Deep Neural Network. (Pre-Lab)

Ans: Backpropagation consists of: 1) Forward pass to compute predictions and loss, 2) Backward pass applying the Calculus Chain Rule from output to input layers to compute error gradients (dL/dw), and 3) Weight update via gradient descent.

Q2. Explain the role of hidden layers in backpropagation. (Post-Lab)

Ans: Hidden layers learn hierarchical intermediate representations: early hidden layers extract low-level features, while deeper hidden layers combine them into high-level abstract patterns.

Q3. Which mathematical rule is the foundation of Backpropagation?

Ans: The Chain Rule of differential calculus: dL/dw_ij = (dL/da_j) * (da_j/dz_j) * (dz_j/dw_ij).

Q4. What is the Vanishing Gradient Problem in Deep Neural Networks?

Ans: When using Sigmoid or Tanh activations in deep networks, derivatives (< 0.25 for sigmoid) multiply across layers during backpropagation, causing gradients in early layers to shrink toward zero and halting learning.

Q5. How does the ReLU activation function solve the Vanishing Gradient Problem?

Ans: ReLU f(x) = max(0, x) has a constant derivative of 1 for all positive inputs (x > 0), allowing gradients to flow backward through many hidden layers without exponential decay.

Q6. What is the Exploding Gradient Problem and how is it mitigated?

Ans: When weights are large, gradients grow exponentially across layers during backpropagation, causing NaN weights. It is mitigated using Gradient Clipping, proper weight initialization (He/Xavier), and Batch Normalization.

Q7. Why must weights in a DNN be initialized randomly rather than all zeros?

Ans: If all weights are initialized to zero (or the same constant), every neuron in a hidden layer computes the exact same output and receives the exact same gradient (symmetry problem), acting like a single neuron.

Q8. What is Dropout regularization in a Deep Neural Network?

Ans: Dropout randomly deactivates a fraction p of neurons during each training step, preventing co-adaptation of neurons and reducing overfitting.

Experiment 8: Design and Implement a CNN Model for Digit Recognition (MNIST)

Aim: *Design and implement a Convolutional Neural Network (CNN) model for handwritten digit recognition.*

Q1. Explain the CNN architecture. (Pre-Lab)

Ans: A CNN consists of: 1) Convolutional layers (feature extraction via learnable filters), 2) Non-linear activation (ReLU), 3) Pooling layers (spatial downsampling), 4) Flatten layer, and 5) Fully Connected (Dense) layers with Softmax for classification.

Q2. Explain the relation between input size, output size, filter size, stride, and padding. (Post-Lab)

Ans: For input size W, filter size F, padding P, and stride S, the spatial output dimension is: O = floor((W - F + 2P) / S) + 1.

Q3. Why are CNNs preferred over Dense (MLP) networks for image recognition?

Ans: CNNs exploit spatial locality and weight sharing (parameter efficiency) and provide translation invariance, whereas flattening a high-resolution image into an MLP loses 2D spatial structure and creates millions of parameters.

Q4. What is the difference between 'valid' padding and 'same' padding?

Ans: 'Valid' padding uses no zero-padding (P = 0), shrinking spatial dimensions. 'Same' padding adds zeros around the border so the output feature map has the same height and width as the input (when stride = 1).

Q5. What is the purpose of MaxPooling2D in a CNN?

Ans: MaxPooling2D takes the maximum value in each local window (e.g., 2x2), reducing spatial dimensions by half, lowering computation/parameters, and providing invariance to small shifts and distortions.

Q6. Why is Softmax used in the final layer of the MNIST digit classifier?

Ans: Softmax converts the 10 raw logit scores (for digits 0–9) into a valid probability distribution where each class probability is in (0,1) and all 10 probabilities sum to 1.0.

Q7. What are the dimensions of the MNIST dataset images?

Ans: MNIST contains 60,000 training and 10,000 testing grayscale images of handwritten digits (0–9), each of size 28 x 28 x 1 pixels.

Q8. Why do we normalize pixel values by dividing by 255.0 before training?

Ans: Scaling pixel intensities from [0, 255] to [0.0, 1.0] keeps inputs small and uniform, stabilizing gradients and speeding up optimizer convergence.

Experiment 9: Design and Implement an Autoencoder Model for Image Compression

Aim: *Design the architecture and implement an Autoencoder model for unsupervised image compression and reconstruction.*

Q1. Explain the use of Autoencoders. (Pre-Lab)

Ans: Autoencoders are unsupervised neural networks used for dimensionality reduction, lossy image compression, image denoising, anomaly detection, and latent feature extraction.

Q2. Explain the different types of Autoencoders. (Post-Lab)

Ans: 1) Undercomplete/Vanilla Autoencoder (bottleneck compression), 2) Denoising Autoencoder (removes noise), 3) Sparse Autoencoder (L1 sparsity penalty), 4) Convolutional Autoencoder, and 5) Variational Autoencoder (VAE - generative).

Q3. What are the three main components of an Autoencoder architecture?

Ans: 1) Encoder (compresses high-dimensional input x into low-dimensional latent code z), 2) Bottleneck / Latent Space (compressed representation z), and 3) Decoder (reconstructs x_hat from z).

Q4. Why is an Autoencoder considered unsupervised (or self-supervised) learning?

Ans: Because it does not require external class labels; the input image X itself serves as the target output (the network trains to minimize reconstruction error between X and X_hat).

Q5. In our MNIST compression experiment, what is the compression ratio achieved?

Ans: Each 28x28 = 784-pixel image is compressed into a bottleneck vector of 32 (or 64) floats, achieving a compression factor of 784 / 32 = 24.5x (or 12.25x for 64).

Q6. Why is Sigmoid used in the Decoder's output layer when inputs are normalized to [0, 1]?

Ans: Sigmoid restricts reconstructed pixel values strictly to the range (0, 1), matching the normalized grayscale input pixel distribution.

Q7. What happens if the bottleneck dimension is larger than the input dimension without regularization?

Ans: The autoencoder becomes 'overcomplete' and may simply learn the identity function (copying input to output without learning meaningful compressed features).

Q8. How does an Autoencoder differ from Principal Component Analysis (PCA)?

Ans: PCA is restricted to linear projections, whereas an Autoencoder with non-linear activation functions can learn complex non-linear manifolds for far superior compression.

Experiment 10: Design and Implement an LSTM / RNN Model for Sentiment Analysis

Aim: *Design and implement a Long Short-Term Memory (LSTM) Recurrent Neural Network model for text sentiment analysis.*

Q1. Explain why Recurrent Neural Networks (RNNs) and LSTMs are used for sequence/text data. (Pre-Lab)

Ans: Unlike feedforward networks and CNNs (which assume independent fixed-size inputs), RNNs and LSTMs maintain an internal memory (hidden/cell state) across time steps to capture sequential order and context in text.

Q2. What problem in standard RNNs does LSTM solve, and how? (Post-Lab)

Ans: Standard RNNs suffer from short-term memory due to vanishing gradients over long sequences. LSTM solves this with a dedicated Cell State (C_t) highway and three multiplicative gates that regulate information flow.

Q3. Name and explain the three gates in an LSTM cell.

Ans: 1) Forget Gate (decides what information to discard from cell state), 2) Input Gate (decides what new candidate information to store in cell state), and 3) Output Gate (decides what part of the cell state to output as hidden state h_t).

Q4. What is the purpose of the Embedding layer in NLP / Sentiment Analysis?

Ans: The Embedding layer maps sparse integer word indices (from a vocabulary of e.g., 10,000 words) into dense, low-dimensional continuous vectors where semantically similar words lie close together.

Q5. Why do we use pad_sequences() when preprocessing IMDB movie reviews?

Ans: Movie reviews have variable word lengths, but neural networks require uniform tensor shapes per batch; pad_sequences() pads shorter reviews with 0s and truncates longer reviews to a fixed maxlen (e.g., 100).

Q6. What is the role of EarlyStopping and ModelCheckpoint callbacks during training?

Ans: EarlyStopping halts training when validation loss stops improving (preventing overfitting), while ModelCheckpoint saves the model weights from the epoch with the best validation performance.

Q7. What is the difference between LSTM and GRU (Gated Recurrent Unit)?

Ans: GRU merges the cell state and hidden state and uses only two gates (Reset and Update gates) instead of three, making it computationally faster with fewer parameters than LSTM.

Q8. Why is Sigmoid with binary_crossentropy used in the final layer of IMDB sentiment analysis?

Ans: Because IMDB sentiment classification is a binary task (0 = Negative review, 1 = Positive review), requiring a single probability output in (0, 1).